Disparadores: código que usted no invoca

Dónde estamos

- Funciones y procedimientos se ejecutan porque alguien los llama. El disparador se ejecuta porque ocurrió un evento, y nadie puede invocarlo a mano.
- Esa diferencia lo hace poderoso y peligroso en la misma medida: la lógica queda fuera de la vista de quien escribe el INSERT.

Qué vamos a hacer

- Clasificar los tipos de disparador y saber exactamente cuándo se dispara cada uno.
- Escribir auditoría y reglas de negocio que no se pueden expresar con restricciones declarativas.
- Entender la tabla mutante, reproducirla y resolverla con un disparador compuesto.

Recorrido

- Taxonomía y orden de disparo. :NEW y :OLD. Predicados condicionales. INSTEAD OF sobre vistas.
- ORA-04091, disparador compuesto, FOLLOWS, disparadores de sistema y diccionario. Taller guiado con el script 04.

Antes de escribir un disparador, descarte las alternativas

Lo que no necesita un disparador

- Valor por omisión de una columna: use DEFAULT en la tabla, o una columna IDENTITY para el consecutivo.
- Unicidad, obligatoriedad, rango simple o integridad referencial: use restricciones declarativas, que el motor optimiza y documenta.
- Columna calculada a partir de otras de la misma fila: use una columna virtual.

Lo que sí justifica un disparador

- Auditoría de quién cambió qué y cuándo, cuando la aplicación no puede garantizarlo.
- Reglas que cruzan varias tablas o varias filas y no se pueden expresar de forma declarativa.
- Hacer actualizable una vista compleja, mediante INSTEAD OF.

El costo que se asume

- El disparador corre dentro de la transacción del usuario: su tiempo se suma al de cada DML y sus bloqueos también.
- La documentación de Oracle es explícita en que los disparadores no son un mecanismo de seguridad: se pueden deshabilitar.

Taxonomía: cuatro tipos y un criterio

Tipo	Evento que lo dispara	Cuándo se usa
DML simple	INSERT, UPDATE o DELETE sobre una tabla	Auditoría, valores derivados, validación entre tablas
INSTEAD OF	DML sobre una vista	Hacer actualizable una vista que el motor no puede resolver
Compuesto (compound)	DML, con secciones para cada momento	Compartir estado entre filas y evitar la tabla mutante
De sistema o DDL	CREATE, ALTER, DROP, LOGON, LOGOFF, SERVERERROR	Auditoría de esquema y control de sesiones

Dos dimensiones más que se combinan con el tipo
El momento: BEFORE o AFTER. Y el alcance: FOR EACH ROW, que se dispara una vez por fila afectada, o de sentencia, que se dispara una sola vez aunque el DML toque mil filas.

Orden de disparo sobre una misma tabla

Orden	Qué se ejecuta	Se ejecuta
1	BEFORE de sentencia	Una vez, antes de tocar la primera fila
2	BEFORE de fila	Una vez por cada fila afectada
3	El cambio en la fila y la verificación de restricciones	Por cada fila
4	AFTER de fila	Una vez por cada fila afectada
5	AFTER de sentencia	Una vez, cuando ya terminó todo el DML

Consecuencia práctica
Modificar :NEW solo tiene efecto en un disparador BEFORE de fila; en un AFTER la fila ya se escribió. Y una regla que necesita ver el resultado completo del DML pertenece a un AFTER de sentencia.

:NEW y :OLD según el evento

Evento	:OLD	:NEW
INSERT	Todos los campos en NULL	Los valores que se van a insertar
UPDATE	Los valores anteriores al cambio	Los valores que quedarán
DELETE	Los valores que se van a borrar	Todos los campos en NULL

```
-- en el cuerpo del disparador llevan dos puntos; en la cláusula WHEN, no
CREATE OR REPLACE TRIGGER trg_nota_audita
BEFORE UPDATE OF nota ON trg_inscripcion FOR EACH ROW
WHEN (NEW.nota <> OLD.nota)          -- sin los dos puntos
BEGIN
  :NEW.usuario_modifica := USER;     -- con los dos puntos
END;
```

Auditoría con predicados condicionales

```
CREATE OR REPLACE TRIGGER trg_inscripcion_audita
AFTER INSERT OR UPDATE OR DELETE ON trg_inscripcion
FOR EACH ROW
DECLARE
  v_operacion VARCHAR2(10);
BEGIN
  IF    INSERTING THEN v_operacion := 'INSERT';
  ELSIF UPDATING  THEN v_operacion := 'UPDATE';
  ELSIF DELETING  THEN v_operacion := 'DELETE';
  END IF;

  INSERT INTO trg_auditoria (fecha, usuario, operacion, id_anterior, id_nuevo)
       VALUES (SYSTIMESTAMP, USER, v_operacion, :OLD.id_inscripcion, :NEW.id_inscripcion);
END;
```

Un solo disparador para los tres eventos evita triplicar la lógica, pero exige cuidado: en DELETE, :NEW está vacío.

Valores derivados: primero pregunte si hace falta

```sql
-- disparador BEFORE INSERT
CREATE OR REPLACE TRIGGER trg_inscripcion_bi
BEFORE INSERT ON trg_inscripcion FOR EACH ROW
BEGIN
  :NEW.fecha_registro := SYSDATE;
  :NEW.estado := NVL(:NEW.estado, 'PENDIENTE');
END;
```

```sql
-- la alternativa declarativa, desde 12c
CREATE TABLE trg_inscripcion (
  id_inscripcion NUMBER GENERATED ALWAYS
                 AS IDENTITY,
  fecha_registro DATE DEFAULT SYSDATE,
  estado VARCHAR2(12) DEFAULT 'PENDIENTE');
```

Cuál conviene

La versión declarativa es más rápida, se documenta sola en el diccionario y no se puede deshabilitar por accidente. El disparador se justifica cuando el valor depende de una condición que DEFAULT no puede expresar.

INSTEAD OF: hacer actualizable una vista

```
CREATE OR REPLACE VIEW v_inscripcion_detalle AS
  SELECT i.id_inscripcion, i.estado, e.nombre, a.titulo
    FROM trg_inscripcion i, trg_estudiante e, trg_asignatura a
   WHERE i.id_estudiante = e.id_estudiante AND i.id_asignatura = a.id_asignatura;

CREATE OR REPLACE TRIGGER trg_v_inscripcion_upd
INSTEAD OF UPDATE ON v_inscripcion_detalle
FOR EACH ROW
BEGIN
  UPDATE trg_inscripcion SET estado = :NEW.estado
   WHERE id_inscripcion = :OLD.id_inscripcion;   -- se traduce a la tabla base
END;
```

Regla del tipo
INSTEAD OF solo existe sobre vistas y siempre es FOR EACH ROW. Reemplaza al DML original: si el cuerpo no hace nada, la sentencia se ejecuta sin error y sin efecto.

La tabla mutante: ORA-04091

Un disparador de fila no puede consultar ni modificar la tabla que está siendo alterada por la sentencia que lo disparó

```
CREATE OR REPLACE TRIGGER trg_tope_cupo
BEFORE INSERT ON trg_inscripcion FOR EACH ROW
DECLARE v_total NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_total            -- lee la misma tabla que se modifica
    FROM trg_inscripcion WHERE id_asignatura = :NEW.id_asignatura;
  IF v_total >= 30 THEN
     RAISE_APPLICATION_ERROR(-20020, 'Cupo lleno');
  END IF;
END;
-- ORA-04091: table TRG_INSCRIPCION is mutating, trigger/function may not see it
```

No es un defecto del motor: es la garantía de que el disparador no vea un estado intermedio e inconsistente de la tabla.

El disparador compuesto resuelve el caso

```
CREATE OR REPLACE TRIGGER trg_tope_cupo
FOR INSERT ON trg_inscripcion
COMPOUND TRIGGER
  TYPE t_asig IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  v_asig t_asig;                        -- estado compartido entre secciones

  AFTER EACH ROW IS BEGIN
    v_asig(:NEW.id_asignatura) := 1;    -- solo recolecta, no consulta
  END AFTER EACH ROW;

  AFTER STATEMENT IS BEGIN              -- aquí la tabla ya no muta
    NULL;  -- validar el cupo de cada asignatura recolectada
  END AFTER STATEMENT;
END trg_tope_cupo;
```

Por qué funciona

Las cuatro secciones comparten variables durante toda la sentencia, y en AFTER STATEMENT la tabla ya terminó de cambiar. Antes de 11g esto exigía tres objetos: un paquete, un disparador de fila y uno de sentencia.

Varios disparadores sobre el mismo evento

```
CREATE OR REPLACE TRIGGER trg_inscripcion_valida
BEFORE INSERT ON trg_inscripcion FOR EACH ROW
FOLLOWS trg_inscripcion_bi          -- se ejecuta después de aquel
BEGIN NULL; END;
/
ALTER TRIGGER trg_inscripcion_valida DISABLE;   -- suspender sin borrar
ALTER TABLE  trg_inscripcion DISABLE ALL TRIGGERS;
```

Situación	Qué hace Oracle
Dos disparadores para el mismo evento y momento	El orden no está definido, salvo que se declare FOLLOWS o PRECEDES
Un disparador deshabilitado	El objeto sigue existiendo y compilado, pero no se dispara
Un disparador con error de compilación	El DML sobre la tabla falla con ORA-04098

Disparadores de sistema y de DDL

```
CREATE OR REPLACE TRIGGER trg_audita_ddl
AFTER CREATE OR ALTER OR DROP ON SCHEMA
BEGIN
  INSERT INTO trg_auditoria_ddl (fecha, usuario, evento, objeto, tipo)
       VALUES (SYSTIMESTAMP, ORA_LOGIN_USER, ORA_SYSEVENT,
               ORA_DICT_OBJ_NAME, ORA_DICT_OBJ_TYPE);
END;
/
```

Atributos del evento

Los atributos ORA_SYSEVENT, ORA_DICT_OBJ_NAME, ORA_LOGIN_USER y los demás vienen del paquete estándar y solo tienen valor dentro de un disparador de sistema. Sobre DATABASE requieren privilegio de administración.

En un banco o una entidad pública, este disparador suele ser la primera evidencia de que alguien alteró un objeto fuera de ventana.

Diccionario y diagnóstico

```
SELECT trigger_name, trigger_type, triggering_event, status
  FROM user_triggers WHERE table_name = 'TRG_INSCRIPCION';

SELECT trigger_body FROM user_triggers            -- el cuerpo almacenado
 WHERE trigger_name = 'TRG_INSCRIPCION_AUDITA';

SELECT name, line, text FROM user_errors          -- errores de compilación
 WHERE type = 'TRIGGER';

ALTER TRIGGER trg_inscripcion_audita COMPILE;
```

Antes de tocar una tabla heredada, consulte USER_TRIGGERS: casi siempre hay lógica escondida que nadie documentó.

Errores frecuentes con disparadores

Síntoma	Causa típica	Corrección
ORA-04091 tabla mutante	Disparador de fila que consulta su propia tabla	Disparador compuesto, o mover la regla a un procedimiento
ORA-04098 disparador inválido	El disparador no compiló y el DML no puede continuar	Revisar USER_ERRORS y recompilar
El valor asignado a :NEW no queda	Se asignó en un disparador AFTER de fila	Mover la asignación a un BEFORE de fila
ORA-04092 no se puede confirmar	COMMIT o ROLLBACK dentro del disparador	Quitarlo, o aislarlo en una transacción autónoma si es auditoría
El DML se volvió lento sin causa aparente	Disparador de fila con consultas dentro, por cada fila	Pasar a disparador de sentencia o a proceso masivo

Cómo trabajar el script 04 paso a paso

Antes de ejecutar

- Abra 04_triggers_plsql19c.sql con un usuario que tenga CREATE TABLE, CREATE TRIGGER y CREATE VIEW en su propio esquema.
- Ejecute SET SERVEROUTPUT ON. El script crea su propio esquema con el prefijo TRG_ y no depende de los otros módulos.

Durante la práctica

- Los primeros pasos construyen la auditoría y muestran el orden real de disparo con mensajes en pantalla.
- El paso de tabla mutante debe fallar con ORA-04091. Léalo completo antes de ejecutar el paso siguiente, que lo resuelve.
- El paso de INSTEAD OF pide actualizar la vista y luego verificar la tabla base: ahí se comprueba que la traducción funcionó.

Al terminar

- Resuelva los cinco ejercicios propuestos; cada uno trae pista y criterio de logrado.
- El paso final deshabilita y elimina los disparadores antes de borrar las tablas, en ese orden.

Lo que hay que llevarse

Del criterio

- Antes de escribir un disparador, agote las restricciones declarativas, los valores por omisión y las columnas virtuales.
- La lógica en disparadores es invisible para quien escribe el INSERT: documéntela o alguien la descubrirá depurando de madrugada.
- La tabla mutante no es un obstáculo que esquivar: avisa que la regla no pertenece a un disparador de fila.

Del oficio

- BEFORE para modificar la fila, AFTER para reaccionar a ella, de sentencia para razonar sobre el conjunto.
- Sin COMMIT ni ROLLBACK dentro del disparador, y consulte USER_TRIGGERS antes de intervenir una tabla ajena.

Lecturas para cerrar el ciclo

- PL/SQL Language Reference 19c, capítulo PL/SQL Triggers: tipos, orden de disparo y disparadores compuestos.
- Database Development Guide 19c, capítulo sobre uso de disparadores y alternativas declarativas.
- docs.oracle.com/en/database/oracle/oracle-database/19/lnpls